Guess X - PRISMA_BETTER_AUTH_MIGRATION

- 1 Prisma + Better-auth Migration Guide
  - 1.1 Overview
  - 1.2 Architecture
    - 1.2.1 Dual ORM Setup (Gradual Migration)
  - 1.3 What Was Added
    - 1.3.1 1. Prisma Schema (`prisma/schema.prisma`)
    - 1.3.2 2. Prisma Service (`src/prisma/`)
    - 1.3.3 3. Better-auth Configuration (`src/auth/`)
  - 1.4 Usage
    - 1.4.1 Using Prisma
    - 1.4.2 Using Better-auth
  - 1.5 Commands
    - 1.5.1 Prisma Commands
    - 1.5.2 TypeORM Commands (Legacy - still available)
  - 1.6 Migration Strategy
    - 1.6.1 Phase 1: Dual Setup (Current)
    - 1.6.2 Phase 2: Gradual Service Migration
    - 1.6.3 Phase 3: Complete Migration
  - 1.7 Files Added
    - 1.7.1 identity-service
    - 1.7.2 auction-engine
    - 1.7.3 query-service
  - 1.8 Dependencies Added
  - 1.9 Verification

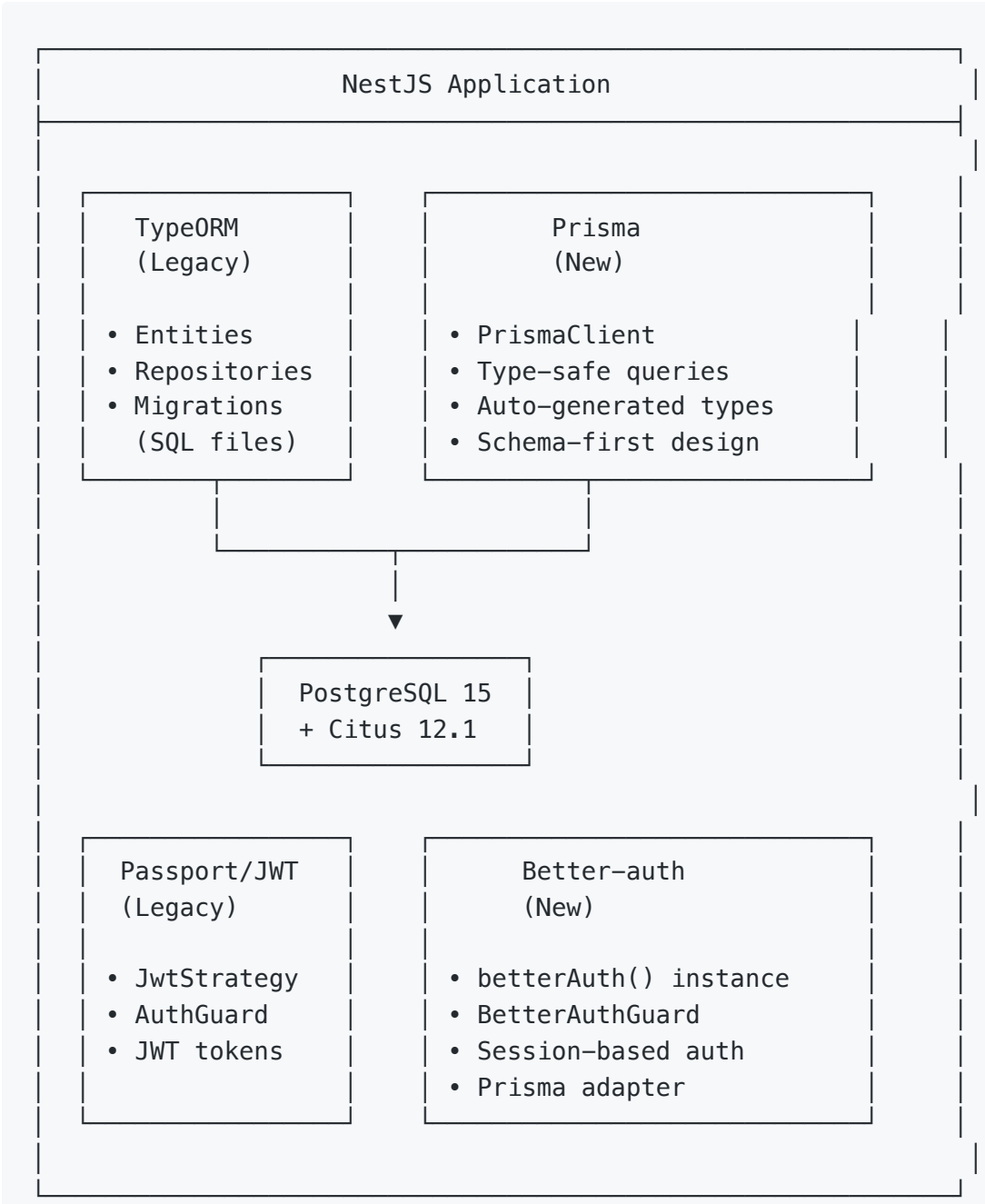
1 Prisma + Better-auth Migration Guide

1.1 Overview

Guess X has been upgraded to support **Prisma ORM** (alongside TypeORM) and **Better-auth** (alongside Passport/JWT). This document describes the new architecture and how to use it.

1.2 Architecture

1.2.1 Dual ORM Setup (Gradual Migration)



1.3 What Was Added

1.3.1 1. Prisma Schema (`prisma/schema.prisma`)

All 3 services now have a complete Prisma schema that maps to the existing PostgreSQL database:

- **13 models**: User, Product, Auction, Bid, Winner, Transaction, PaymentTransaction, Favorite, NotificationLog, AuditLog, UserPermission, Otp
- **7 enums**: AuctionStatus, TransactionType, PaymentGateway, PaymentTransactionStatus, PaymentType, AuthProvider, UserRole
- **All relations**: Foreign keys, cascade rules, unique constraints
- **All indexes**: Matching the existing database indexes

1.3.2 2. Prisma Service (`src/prisma/`)

Each service has a `PrismaModule` and `PrismaService`:

```
// src/prisma/prisma.service.ts
@Injectable()
export class PrismaService extends PrismaClient implements OnModuleInit,
        OnModuleDestroy {
  async onModuleInit() { await this.$connect(); }
  async onModuleDestroy() { await this.$disconnect(); }
}

// src/prisma/prisma.module.ts
@Global()
@Module({
  providers: [PrismaService],
  exports: [PrismaService],
})
export class PrismaModule {}
```

1.3.3 3. Better-auth Configuration (`src/auth/`)

The identity-service has a complete Better-auth setup:

- `better-auth.config.ts` : Main auth configuration with Prisma adapter, bcrypt hashing, session management
- `better-auth.service.ts` : NestJS service wrapper for auth operations
- `better-auth.guard.ts` : NestJS guard replacing Passport JWT guard
- `better-auth.module.ts` : NestJS module exporting auth services

1.4 Usage

1.4.1 Using Prisma

```
// Inject PrismaService in any service
@Injectable()
```

```
export class MyService {
  constructor(private prisma: PrismaService) {}

  async getUsers() {
    return this.prisma.user.findMany({
      include: { bids: true, transactions: true },
    });
  }

  async createUser(data: { phone_number: string; full_name: string }) {
    return this.prisma.user.create({ data });
  }

  async getActiveAuctions() {
    return this.prisma.auction.findMany({
      where: { status: 'ACTIVE' },
      include: { product: true, bids: true },
    });
  }
}
```

1.4.2 Using Better-auth

```
// In a controller
@Controller('auth')
export class AuthController {
  constructor(private betterAuth: BetterAuthService) {}

  @Post('login')
  async login(@Body() dto: LoginDto) {
    return this.betterAuth.signIn(dto.email, dto.password);
  }

  @Post('register')
  async register(@Body() dto: RegisterDto) {
    return this.betterAuth.signUp(dto.email, dto.password, dto.full_name);
  }
}

// Using the guard
@UseGuards(BetterAuthGuard)
@Get('profile')
async getProfile(@Req() req: any) {
  return req.user;
}
```

1.5 Commands

1.5.1 Prisma Commands

```
# Generate Prisma Client (after schema changes)
npm run prisma:generate

# Create and apply migration (development)
npm run prisma:migrate

# Deploy migrations (production)
npm run prisma:deploy

# Open Prisma Studio (database GUI)
npm run prisma:studio

# Seed database
npm run prisma:seed
```

1.5.2 TypeORM Commands (Legacy - still available)

```
# Run existing SQL migrations
npm run migration:run

# Generate TypeORM migration
npm run migration:generate

# Revert last migration
npm run migration:revert
```

1.6 Migration Strategy

1.6.1 Phase 1: Dual Setup (Current)

- ✅ Prisma installed alongside TypeORM
- ✅ Better-auth installed alongside Passport/JWT
- ✅ All existing tests pass
- ✅ Prisma schema matches database
- ✅ PrismaService available in all services

1.6.2 Phase 2: Gradual Service Migration

- Replace TypeORM repositories with Prisma queries in each service
- Replace Passport JWT guards with BetterAuthGuard
- Update tests to use Prisma client mocks

1.6.3 Phase 3: Complete Migration

- Remove TypeORM dependencies
- Remove Passport/JWT dependencies
- Remove TypeORM entities and config
- Update all tests

1.7 Files Added

1.7.1 identity-service

- `prisma/schema.prisma` - Complete database schema
- `prisma.config.ts` - Prisma configuration
- `src/prisma/prisma.service.ts` - Prisma client service
- `src/prisma/prisma.module.ts` - Global Prisma module
- `src/auth/better-auth.config.ts` - Better-auth configuration
- `src/auth/better-auth.service.ts` - Auth service wrapper
- `src/auth/better-auth.guard.ts` - Auth guard
- `src/auth/better-auth.module.ts` - Auth module

1.7.2 auction-engine

- `prisma/schema.prisma` - Complete database schema
- `prisma.config.ts` - Prisma configuration
- `src/prisma/prisma.service.ts` - Prisma client service
- `src/prisma/prisma.module.ts` - Global Prisma module

1.7.3 query-service

- `prisma/schema.prisma` - Complete database schema
- `prisma.config.ts` - Prisma configuration
- `src/prisma/prisma.service.ts` - Prisma client service
- `src/prisma/prisma.module.ts` - Global Prisma module

1.8 Dependencies Added

```
{
  "prisma": "^7.9.1",
  "@prisma/client": "^7.9.1",
  "better-auth": "^1.7.1",
  "@better-auth/prisma-adapter": "^1.7.1"
}
```

1.9 Verification

- ✅ All 65 backend tests pass (54 auction-engine + 8 identity-service + 3 query-service)
- ✅ TypeScript strict mode passes for all 5 projects
- ✅ No breaking changes to existing functionality
- ✅ Prisma client generates successfully for all services
- ✅ Better-auth configuration compiles without errors